

“NexPlay”

- Entertainment Discovery, Branding & Marketing Platform

Software Requirements Specification

Prepared by

Student ID	Name
22301590	Fahim Shahriar Nur
23101067	Nowshin Zahan
22301550	Ayesha Mahjabin
23101504	Nuzhat Nueree

“NexPlay”.....	1
- Entertainment Discovery, Branding & Marketing Platform.....	1
1. Introduction.....	5
1.1 Purpose.....	5
1.2 Scope.....	6
1.3 Definitions, Acronyms and Abbreviations.....	6
1.4 References.....	8
1.5 Overview.....	8
2. Overall Description.....	9
2.1 Product Perspective.....	9
2.2 Product Features.....	9
Company Management.....	9
Entertainment Discovery.....	10
Streaming Platform Integration.....	10
Sports and Community Features.....	10
2.3 User Classes and Characteristics.....	10
General Users.....	10
Entertainment Companies.....	10
Content Moderators.....	11
Administrators.....	11
2.4 Operating Environment.....	11
Client Environment.....	11
Server Environment.....	11
Database.....	11
Hosting Environment.....	11
External Services.....	12
2.5 Constraints.....	12
2.6 Assumptions and Dependencies.....	12
3. System Requirements.....	13
3.1 Functional Requirements.....	13
3.1.1 Company Management.....	13
3.1.1.1 Company Profile Management.....	13
Functional Requirements.....	13
3.1.1.2 Company Verification.....	13
Functional Requirements.....	13
3.1.1.3 Company Dashboard.....	14

Functional Requirements.....	14
3.1.1.4 Advertisement and Campaign Management.....	14
Functional Requirements.....	14
3.1.1.5 Upcoming Content Management.....	14
Functional Requirements.....	15
3.1.2 Entertainment Discovery.....	15
3.1.2.1 Browse Entertainment Content.....	15
Functional Requirements.....	15
3.1.2.2 Search and Advanced Filtering.....	15
Functional Requirements.....	15
3.1.2.3 Trending and Featured Content.....	16
Functional Requirements.....	16
3.1.2.4 Upcoming Release Calendar.....	16
Functional Requirements.....	16
3.1.2.5 Content Details Page.....	16
Functional Requirements.....	16
3.1.3 Streaming Platform Integration.....	17
3.1.3.1 Streaming Platform Directory.....	17
Functional Requirements.....	17
3.1.3.2 "Where to Watch" Feature.....	17
Functional Requirements.....	17
3.1.3.3 Official Streaming Platform Redirect.....	18
Functional Requirements.....	18
3.1.3.4 Watchlist Management.....	18
Functional Requirements.....	18
3.1.3.5 Personalized Recommendations.....	18
Functional Requirements.....	19
3.1.4 Sports & Community Features.....	19
3.1.4.1 Live Sports Dashboard.....	19
Functional Requirements.....	19
3.1.4.2 Live Scores and Match Schedule.....	19
Functional Requirements.....	19
3.1.4.3 Official Sports Streaming Links.....	20
Functional Requirements.....	20
3.1.4.4 Ratings and Reviews.....	20
Functional Requirements.....	20
3.1.4.5 Notification and Announcement System.....	20
Functional Requirements.....	20

3.2 Non-Functional Requirements.....	21
3.2.1 Performance Requirements.....	21
Performance Requirements.....	21
3.2.2 Security Requirements.....	21
Security Requirements.....	21
3.2.3 Reliability and Availability.....	22
Reliability Requirements.....	22
3.2.4 Maintainability.....	22
Maintainability Requirements.....	23
3.2.5 Scalability.....	23
Scalability Requirements.....	23
3.2.6 Usability.....	23
Usability Requirements.....	23
3.2.7 Compatibility.....	24
Compatibility Requirements.....	24
3.3 External Interface Requirements.....	24
3.3.1 User Interface Requirements.....	24
User Interface Requirements.....	24
3.3.2 Hardware Interface Requirements.....	25
Hardware Interface Requirements.....	25
3.3.3 Software Interface Requirements.....	26
Software Interface Requirements.....	26
3.3.4 Communication Interface Requirements.....	26
Communication Interface Requirements.....	26
4. Technology Stack & Architectural Overview.....	27
4.1 Technology Stack.....	27
Frontend.....	27
Backend.....	27
Database.....	27
Authentication.....	27
Version Control.....	27
API Testing.....	27
Third-Party APIs.....	28
Deployment.....	28
4.2 High-Level Architecture (MVC).....	28
View Layer (Presentation Layer).....	28
Controller Layer (Business Logic Layer).....	28
Model Layer (Data Layer).....	29

MVC Request Flow.....	29
5. Challenges.....	29
5.1 Third-Party API Integration.....	30
5.2 Live Sports Data Synchronization.....	30
5.3 Recommendation System.....	30
5.4 Secure Authentication and Authorization.....	30
5.5 Responsive User Interface.....	30
5.6 Scalability.....	30
5.7 Maintaining MVC Architecture.....	30
6. Sprint Plan.....	31
Sprint 1 – Company & Platform Foundation.....	31
Objective.....	31
Features.....	31
Deliverables.....	31
Sprint 2 – Entertainment Discovery.....	31
Objective.....	31
Features.....	32
Deliverables.....	32
Sprint 3 – Streaming Platform Integration.....	32
Objective.....	32
Features.....	32
Deliverables.....	32
Sprint 4 – Sports & Community Features.....	32
Objective.....	33
Features.....	33
Deliverables.....	33
7. Acceptance Criteria.....	33
8. Conclusion.....	34

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements of **NexPlay**, a web-based Entertainment Discovery, Branding, and Marketing Platform.

NexPlay provides a centralized platform where entertainment companies can promote their brands, advertise upcoming movies, TV shows, web series, anime, documentaries, and live sports events. The platform enables users to discover entertainment content, access official streaming platforms, receive personalized recommendations, and stay updated with live sports information.

Unlike traditional streaming services, NexPlay does not host or distribute copyrighted content. Instead, it serves as an entertainment discovery platform by providing detailed information about entertainment content and redirecting users to official streaming services through secure links.

1.2 Scope

NexPlay is a web-based application developed using the MERN Stack (MongoDB, Express.js, React.js, and Node.js) following the MVC (Model-View-Controller) architecture.

The platform supports three major user groups:

- Entertainment Companies
- Registered Users
- Administrators

Entertainment companies can create verified company profiles, publish advertisements, manage promotional campaigns, and promote upcoming entertainment content.

Registered users can browse entertainment content, search using advanced filters, create watchlists, receive personalized recommendations, submit ratings and reviews, and access official streaming platforms through secure redirects.

Administrators are responsible for verifying companies, moderating platform content, managing users, monitoring system activities, and maintaining platform security.

The system integrates with external APIs such as TMDB API and Sports APIs to provide accurate entertainment information, trailers, release dates, live sports scores, schedules, and official streaming platform information.

1.3 Definitions, Acronyms and Abbreviations

Term	Description
API	Application Programming Interface used for communication between software systems.
MERN	MongoDB, Express.js, React.js, and Node.js full-stack development framework.
MVC	Model-View-Controller software architecture that separates the application into Model, View, and Controller layers.
JWT	JSON Web Token used for authentication and authorization.
REST API	Representational State Transfer architecture used for communication between client, server, and external services.
JSON	JavaScript Object Notation used for exchanging structured data.
CRUD	Create, Read, Update, and Delete operations performed on database records.
RBAC	Role-Based Access Control used to manage user permissions based on assigned roles.
UI	User Interface through which users interact with the platform.
UX	User Experience that focuses on usability and user satisfaction.

TMDB API	Third-party Movie Database API providing information about movies and television shows.
Sports API	External API providing live sports scores, schedules, tournament information, and match updates.
HTTPS	Hypertext Transfer Protocol Secure used for encrypted communication.
SSL/TLS	Security protocols that provide encrypted communication over networks.
MongoDB Atlas	Cloud-hosted MongoDB database service.

1.4 References

The following technologies, frameworks, and official documentation are referenced during the design and development of NexPlay.

- [MongoDB](#)
- [Express.js](#)
- [React](#)
- [Node.js](#)
- [Git](#)
- [GitHub](#)
- [Postman](#)
- [TMDB](#)

1.5 Overview

The remainder of this Software Requirements Specification is organized as follows.

Section 2 describes the overall system, including the product perspective, major features, user classes, operating environment, constraints, and assumptions.

Section 3 specifies all functional requirements, non-functional requirements, and external interface requirements of the NexPlay platform.

Section 4 presents the technology stack and explains the high-level MVC architecture adopted for system development.

Section 5 discusses the technical and project-related challenges expected during development.

Section 6 presents the Agile sprint plan, including sprint objectives, features, and deliverables.

Section 7 defines the acceptance criteria used to evaluate the successful completion of the project.

Section 8 concludes the document by summarizing the objectives, implementation approach, and expected outcomes of the NexPlay platform.

2. Overall Description

2.1 Product Perspective

NexPlay is a standalone web-based Entertainment Discovery, Branding, and Marketing Platform developed using the MERN Stack (MongoDB, Express.js, React.js, and Node.js). The platform serves as a centralized hub where entertainment companies, production houses, streaming platforms, broadcasters, and media organizations can promote their brands, advertise upcoming entertainment content, and connect with a broader audience.

Unlike conventional streaming platforms, NexPlay does not host or stream copyrighted content. Instead, it provides users with comprehensive entertainment information and securely redirects them to official streaming platforms where the selected content is legally available.

The platform integrates with external APIs such as TMDB API and Sports APIs to retrieve up-to-date information regarding movies, TV shows, trailers, cast members, release dates, live sports scores, match schedules, and streaming availability.

The application follows the Model-View-Controller (MVC) architectural pattern, ensuring modularity, maintainability, scalability, and separation of concerns. The responsive web interface allows users to access the platform from desktops, laptops, tablets, and smartphones.

2.2 Product Features

The major features of NexPlay are listed below.

Company Management

- Entertainment companies can create and manage verified company profiles.
- Companies can upload logos, descriptions, official websites, and social media links.
- Companies can publish promotional advertisements and marketing campaigns.
- Companies can manage information regarding upcoming entertainment content.

Entertainment Discovery

- Users can browse movies, TV series, web series, anime, documentaries, and sports content.
- Advanced search functionality allows users to search by title.
- Multiple filters such as genre, language, release year, streaming platform, and popularity help users quickly discover entertainment content.
- Trending and featured entertainment content is displayed dynamically.
- Upcoming releases are presented through an interactive calendar.

Streaming Platform Integration

- Users can view official streaming platforms where selected content is available.
- Secure redirects take users to authorized streaming services.
- Registered users can maintain personal watchlists.
- Personalized recommendations are generated based on user interests and watchlist history.

Sports and Community Features

- Live sports scores and match schedules are displayed using third-party Sports APIs.
- Users can access official sports broadcasting links.
- Users can submit ratings and reviews for entertainment content.
- Notifications keep users informed about upcoming releases, sports events, and promotional campaigns.

2.3 User Classes and Characteristics

General Users

General users use the platform to discover entertainment content, browse categories, search using filters, maintain personal watchlists, submit ratings and reviews, and receive personalized recommendations. The interface is designed to be intuitive and easy to use regardless of the user's technical background.

Entertainment Companies

Entertainment companies include production houses, streaming services, broadcasters, and media organizations. These users manage company profiles, publish advertisements, promote upcoming releases, and monitor audience engagement through their dashboard.

Content Moderators

Content moderators review entertainment company registrations, verify submitted content, moderate advertisements, remove inappropriate reviews, and ensure that published information follows platform policies.

Administrators

Administrators have complete control over the platform. They manage users, verify entertainment companies, monitor advertisements, maintain system security, generate analytical reports, configure system settings, and oversee platform operations.

2.4 Operating Environment

The NexPlay platform operates in the following environment.

Client Environment

- Modern web browsers include Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari.
- Responsive interface supporting desktops, laptops, tablets, and smartphones.

Server Environment

- Node.js runtime environment.
- Express.js web application framework.
- RESTful API architecture.
- MVC software architecture.

Database

- MongoDB as the primary NoSQL database.
- MongoDB Atlas for cloud database deployment and backup.

Hosting Environment

- React frontend hosted on Vercel.
- Backend services deployed on Render or Railway.
- MongoDB Atlas for cloud database management.
- Alternative deployment support using AWS, Microsoft Azure, or Google Cloud Platform.

External Services

- TMDB API for entertainment information.
- Sports API for live sports scores and schedules.
- Official streaming platform links for secure content redirection.

2.5 Constraints

The following constraints apply to the development of NexPlay.

- The application must be developed using the MERN Stack.
- The system shall follow the MVC architectural pattern.
- The platform must comply with the CSE470 Software Engineering project guidelines.
- Communication between client and server must use HTTPS.
- Passwords must be securely hashed before storage.
- Role-Based Access Control (RBAC) shall be implemented.
- Features depending on external APIs are subject to API availability and limitations.
- The platform shall provide acceptable performance under normal user traffic.
- Development will be completed by a team of four members using Agile methodology within four planned sprints.
- GitHub will be used for source code management and collaboration.

2.6 Assumptions and Dependencies

The successful operation of NexPlay depends on the following assumptions.

- Users have stable internet connectivity.
- Entertainment companies provide accurate promotional information.
- External APIs remain available and return reliable data.
- Official streaming platforms maintain valid redirect URLs.
- Users access the application using modern browsers with JavaScript enabled.
- Cloud hosting services and MongoDB Atlas remain operational.

- Third-party APIs continue providing movie information, sports updates, trailers, release dates, and streaming availability.
- Future enhancements can be incorporated without major architectural modifications due to the modular MVC design.

3. System Requirements

3.1 Functional Requirements

This section describes the functional requirements of the NexPlay platform. The requirements are grouped into major functional modules.

3.1.1 Company Management

3.1.1.1 Company Profile Management

The platform shall provide entertainment companies with facilities to establish and maintain their public presence.

Functional Requirements

FR-1: The system shall allow verified entertainment companies to create public company profiles.

FR-2: The system shall allow companies to upload and manage company logos, descriptions, official websites, and social media links.

FR-3: Companies shall be able to edit and update their profile information at any time.

FR-4: The system shall display company profiles publicly after administrator approval.

3.1.1.2 Company Verification

To ensure authenticity, every company must complete the verification process before accessing promotional services.

Functional Requirements

FR-5: The system shall allow administrators to review company registration requests.

FR-6: Administrators shall be able to approve or reject company verification requests.

FR-7: The system shall notify companies regarding their verification status.

FR-8: Only verified companies shall be allowed to publish advertisements and promotional campaigns.

3.1.1.3 Company Dashboard

Each verified company shall receive a dedicated dashboard to manage platform activities.

Functional Requirements

FR-9: The system shall provide a personalized dashboard for every verified company.

FR-10: Companies shall be able to manage company information from the dashboard.

FR-11: Companies shall be able to monitor advertisement performance using basic analytics.

FR-12: Companies shall be able to manage promotional campaigns and upcoming content from a single interface.

3.1.1.4 Advertisement and Campaign Management

Entertainment companies shall be able to promote their brands through advertisements and campaigns.

Functional Requirements

FR-13: Companies shall be able to create promotional advertisements.

FR-14: Companies shall be able to edit scheduled advertisements.

FR-15: Companies shall be able to remove advertisements whenever necessary.

FR-16: The system shall display active advertisements on designated sections of the platform.

3.1.1.5 Upcoming Content Management

Companies shall be able to promote upcoming entertainment releases.

Functional Requirements

FR-17: Companies shall be able to publish information about upcoming movies, TV series, web series, documentaries, anime, and sports events.

FR-18: The system shall display posters, trailers, genres, release dates, and descriptions for published content.

FR-19: Companies shall be able to edit or remove upcoming content before release.

FR-20: The system shall automatically categorize upcoming content based on its entertainment type.

3.1.2 Entertainment Discovery

3.1.2.1 Browse Entertainment Content

Users shall be able to explore entertainment content through organized categories.

Functional Requirements

FR-21: Users shall be able to browse entertainment content by category.

FR-22: Categories shall include Movies, TV Series, Web Series, Anime, Documentaries, and Sports.

FR-23: Users shall be able to browse recently added content.

FR-24: The homepage shall display featured entertainment collections.

3.1.2.2 Search and Advanced Filtering

The platform shall provide efficient searching and filtering capabilities.

Functional Requirements

FR-25: Users shall be able to search entertainment content by title.

FR-26: Users shall be able to filter content by genre.

FR-27: Users shall be able to filter content by language.

FR-28: Users shall be able to filter content by release year, streaming platform, and popularity.

3.1.2.3 Trending and Featured Content

The system shall recommend popular entertainment content.

Functional Requirements

FR-29: The system shall display trending entertainment content.

FR-30: Administrators shall manage featured content shown on the homepage.

FR-31: Trending content shall update dynamically based on popularity.

FR-32: Promotional campaigns may appear within featured content sections.

3.1.2.4 Upcoming Release Calendar

The platform shall provide an organized calendar of upcoming releases.

Functional Requirements

FR-33: The system shall display upcoming releases in calendar format.

FR-34: Users shall filter releases by month.

FR-35: Users shall filter releases by entertainment category.

FR-36: Release dates shall update automatically whenever content information changes.

3.1.2.5 Content Details Page

Each entertainment item shall have a dedicated information page.

Functional Requirements

FR-37: The system shall display posters, trailers, cast information, genres, synopsis, and release dates.

FR-38: Streaming availability information shall be displayed when available.

FR-39: Ratings and reviews shall be visible on the content details page.

FR-40: Users shall be able to add content directly to their watchlists from the details page.

3.1.3 Streaming Platform Integration

3.1.3.1 Streaming Platform Directory

The platform shall maintain a directory of supported streaming services where entertainment content is officially available.

Functional Requirements

FR-41: The system shall maintain a directory of supported streaming platforms.

FR-42: Users shall be able to browse streaming platforms and their available entertainment content.

FR-43: The system shall display platform information including name, logo, website, and supported regions (where available).

FR-44: Administrators shall be able to add, update, or remove supported streaming platforms.

3.1.3.2 "Where to Watch" Feature

The platform shall help users identify where entertainment content can be legally viewed.

Functional Requirements

FR-45: The system shall display all available streaming platforms for a selected movie or TV show.

FR-46: The system shall indicate whether the content is available through subscription, rental, purchase, or free viewing when such information is available.

FR-47: Users shall be able to compare streaming platform availability before selecting a provider.

FR-48: The platform shall update streaming availability whenever new information is received from external APIs.

3.1.3.3 Official Streaming Platform Redirect

NexPlay shall redirect users only to authorized streaming providers.

Functional Requirements

FR-49: Users shall be able to access official streaming platforms through secure redirect links.

FR-50: The platform shall never host or stream copyrighted entertainment content directly.

FR-51: Redirect links shall open in a new browser tab.

FR-52: Invalid or unavailable streaming links shall display an appropriate notification to the user.

3.1.3.4 Watchlist Management

Registered users shall be able to maintain personalized watchlists.

Functional Requirements

FR-53: Users shall be able to add entertainment content to their watchlists.

FR-54: Users shall be able to remove items from their watchlists.

FR-55: Watchlists shall be synchronized with user accounts.

FR-56: Users shall be able to view their complete watchlists from their profile dashboard.

3.1.3.5 Personalized Recommendations

The system shall recommend entertainment content based on user interests.

Functional Requirements

FR-57: The system shall recommend entertainment content based on watchlists.

FR-58: Recommendations shall consider browsing history and user preferences.

FR-59: Recommendation lists shall be updated periodically.

FR-60: Users shall receive personalized recommendations on their homepage.

3.1.4 Sports & Community Features

3.1.4.1 Live Sports Dashboard

The platform shall provide live sports information for supported tournaments.

Functional Requirements

FR-61: The system shall display live sports events.

FR-62: Users shall browse sports by category.

FR-63: Users shall browse tournaments and competitions.

FR-64: Live sports information shall be updated automatically through external Sports APIs.

3.1.4.2 Live Scores and Match Schedule

The system shall provide real-time sports information.

Functional Requirements

FR-65: The system shall display live scores.

FR-66: The system shall display upcoming match schedules.

FR-67: Completed match results shall remain accessible for users.

FR-68: Sports information shall be retrieved through third-party Sports APIs.

3.1.4.3 Official Sports Streaming Links

Users shall be able to access authorized sports broadcasters.

Functional Requirements

FR-69: The system shall provide verified streaming links for live sports events.

FR-70: Users shall be redirected only to official broadcasters.

FR-71: Streaming availability shall be displayed whenever available.

FR-72: The platform shall not host live sports streams directly.

3.1.4.4 Ratings and Reviews

The platform shall encourage community participation through ratings and reviews.

Functional Requirements

FR-73: Registered users shall be able to submit ratings.

FR-74: Registered users shall be able to submit reviews.

FR-75: The system shall display average ratings for entertainment content.

FR-76: Administrators shall be able to moderate inappropriate reviews.

3.1.4.5 Notification and Announcement System

The platform shall notify users about important updates.

Functional Requirements

FR-77: Users shall receive notifications about upcoming entertainment releases.

FR-78: Users shall receive notifications regarding live sports events.

FR-79: Entertainment companies shall be able to publish promotional announcements.

FR-80: Administrators shall be able to broadcast system-wide announcements and important platform updates.

3.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational characteristics of the NexPlay platform. These requirements ensure that the system is efficient, secure, reliable, scalable, and easy to maintain.

3.2.1 Performance Requirements

The system shall provide a fast and responsive user experience under normal operating conditions.

Performance Requirements

NFR-1: The system shall support at least **500 concurrent users** without significant degradation in performance.

NFR-2: The average page load time shall not exceed **3 seconds** under normal network conditions.

NFR-3: Search results shall be returned within **2 seconds** for typical user queries.

NFR-4: Content details pages shall load within **3 seconds**.

NFR-5: API responses for entertainment and sports information shall be optimized to minimize latency.

NFR-6: The system shall efficiently cache frequently accessed data whenever appropriate.

3.2.2 Security Requirements

The system shall ensure the confidentiality, integrity, and availability of user information.

Security Requirements

NFR-7: All communication between the client and server shall use **HTTPS (SSL/TLS)** encryption.

NFR-8: User passwords shall be securely hashed before storage using industry-standard algorithms.

NFR-9: Authentication shall be implemented using **JSON Web Tokens (JWT)**.

NFR-10: The system shall implement **Role-Based Access Control (RBAC)** to restrict system access according to user roles.

NFR-11: The application shall validate all user inputs to prevent malicious data entry.

NFR-12: The system shall protect against common web vulnerabilities including SQL/NoSQL Injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF).

NFR-13: Sensitive information shall never be exposed through API responses.

3.2.3 Reliability and Availability

The platform shall remain operational and recover gracefully from unexpected failures.

Reliability Requirements

NFR-14: The application shall maintain a minimum uptime of **99%**, excluding scheduled maintenance.

NFR-15: The database shall be backed up regularly to minimize data loss.

NFR-16: The system shall gracefully handle failures of third-party APIs.

NFR-17: Users shall receive meaningful error messages whenever external services are temporarily unavailable.

NFR-18: The application shall recover automatically after temporary service interruptions whenever possible.

3.2.4 Maintainability

The software shall be designed to simplify future maintenance and feature development.

Maintainability Requirements

NFR-19: The application shall follow the **MVC (Model-View-Controller)** architectural pattern.

NFR-20: Source code shall follow consistent coding standards and naming conventions.

NFR-21: The project shall maintain clear documentation for APIs, modules, and database schemas.

NFR-22: GitHub shall be used for version control and collaborative development.

NFR-23: Individual system modules shall be loosely coupled to simplify maintenance and future enhancements.

3.2.5 Scalability

The architecture shall support future growth with minimal system modifications.

Scalability Requirements

NFR-24: The application shall support the addition of new entertainment companies without affecting existing functionality.

NFR-25: The database shall efficiently support increasing numbers of users and entertainment content.

NFR-26: New entertainment categories shall be added without major architectural changes.

NFR-27: Additional third-party APIs shall be integrated with minimal modification to the existing system.

NFR-28: The backend architecture shall support horizontal scaling for future production deployment.

3.2.6 Usability

The platform shall provide an intuitive and user-friendly experience for all user groups.

Usability Requirements

NFR-29: The interface shall be responsive across desktops, laptops, tablets, and smartphones.

NFR-30: Navigation shall remain simple, consistent, and intuitive throughout the application.

NFR-31: Users shall be able to perform common tasks with minimal learning effort.

NFR-32: Important actions shall provide clear confirmation or error messages.

NFR-33: The platform shall follow modern UI/UX design principles to improve accessibility and user satisfaction.

3.2.7 Compatibility

The system shall operate consistently across supported browsers and operating systems.

Compatibility Requirements

NFR-34: The application shall support the latest versions of Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari.

NFR-35: The platform shall function correctly on Windows, macOS, Android, and iOS devices.

NFR-36: The frontend shall adapt automatically to different screen sizes using responsive design techniques.

3.3 External Interface Requirements

External interface requirements define how NexPlay interacts with users, external software systems, hardware devices, databases, and communication services.

3.3.1 User Interface Requirements

The user interface shall provide a modern, responsive, and intuitive experience for all users.

User Interface Requirements

UI-1: The application shall provide a responsive web interface developed using **React.js**.

UI-2: The interface shall adapt automatically to desktops, laptops, tablets, and smartphones.

UI-3: Separate dashboards shall be provided for Administrators, Entertainment Companies, and Registered Users based on their assigned roles.

UI-4: The homepage shall display featured entertainment content, trending releases, advertisements, and live sports updates.

UI-5: Users shall be able to browse entertainment content through categories and interactive navigation menus.

UI-6: The platform shall provide advanced search and filtering options for efficient content discovery.

UI-7: Company dashboards shall allow companies to manage advertisements, promotional campaigns, and upcoming entertainment releases.

UI-8: Administrator dashboards shall provide interfaces for user management, company verification, content moderation, and system monitoring.

UI-9: User dashboards shall display watchlists, personalized recommendations, notifications, and profile information.

UI-10: The interface shall provide meaningful success, warning, and error messages for all user actions.

3.3.2 Hardware Interface Requirements

The application shall operate on commonly available computing devices without requiring specialized hardware.

Hardware Interface Requirements

HI-1: The application shall operate on desktop computers, laptops, tablets, and smartphones.

HI-2: Devices shall require only a modern web browser and an internet connection.

HI-3: Backend services shall execute on cloud servers capable of supporting the MERN stack.

HI-4: Database services shall operate on cloud-hosted MongoDB Atlas servers.

HI-5: No additional hardware peripherals shall be required for normal platform operation.

3.3.3 Software Interface Requirements

NexPlay shall communicate with several internal and external software components.

Software Interface Requirements

SI-1: The system shall integrate with the **TMDB API** (or an equivalent movie database API) to retrieve movie, TV show, cast, trailer, poster, and release information.

SI-2: The system shall integrate with a Sports API to retrieve live sports scores, match schedules, tournament information, and completed match results.

SI-3: The application shall integrate with official streaming platforms to provide authorized viewing links.

SI-4: The backend shall communicate with **MongoDB Atlas** for persistent data storage.

SI-5: User authentication shall utilize JWT-based authentication mechanisms.

SI-6: Email notification services shall be integrated for account verification, announcements, and promotional notifications.

SI-7: RESTful APIs shall be used for communication between the frontend and backend.

3.3.4 Communication Interface Requirements

Communication between all system components shall use secure and standardized protocols.

Communication Interface Requirements

CI-1: The frontend shall communicate with the backend using RESTful APIs.

CI-2: Data exchanged between the frontend and backend shall use JSON format.

CI-3: All communication shall be secured using HTTPS.

CI-4: External API communication shall use secure HTTPS requests.

CI-5: Authentication tokens shall be transmitted securely through HTTP headers.

CI-6: Database communication shall occur over encrypted connections.

CI-7: API responses shall follow standardized HTTP status codes and response structures.

4. Technology Stack & Architectural Overview

4.1 Technology Stack

NexPlay is developed using the **MERN Stack**, providing a modern, scalable, and efficient full-stack web development environment.

Frontend

- **React.js** is used to build a responsive and interactive user interface.
- **React Router** is used for client-side routing.
- **Axios** is used to communicate with backend REST APIs.
- **CSS** and **Bootstrap/Tailwind CSS** are used for responsive UI design.

Backend

- **Node.js** provides the JavaScript runtime environment.
- **Express.js** manages routing, middleware, authentication, and REST API development.

Database

- **MongoDB** stores application data including users, companies, advertisements, entertainment content, watchlists, reviews, and notifications.
- **MongoDB Atlas** provides cloud database hosting.

Authentication

- JSON Web Token (JWT)

Version Control

- Git
- GitHub

API Testing

- Postman

Third-Party APIs

- TMDB API
- Sports API

Deployment

- Frontend: Vercel
- Backend: Render or Railway
- Database: MongoDB Atlas

4.2 High-Level Architecture (MVC)

NexPlay follows the **Model–View–Controller (MVC)** architecture to promote modularity, maintainability, scalability, and separation of concerns.

View Layer (Presentation Layer)

The View layer is implemented using **React.js**.

Responsibilities include:

- Displaying entertainment content
- Rendering dashboards
- Managing user interactions
- Displaying advertisements
- Displaying sports information
- Managing watchlists
- Providing responsive user interfaces

Controller Layer (Business Logic Layer)

The Controller layer is implemented using **Node.js** and **Express.js**.

Responsibilities include:

- Processing user requests
- Validating input data
- Managing authentication and authorization
- Handling CRUD operations
- Communicating with external APIs
- Managing business rules
- Returning API responses

Model Layer (Data Layer)

The Model layer is implemented using **MongoDB**.

Responsibilities include:

- Managing user accounts
- Managing entertainment company information
- Managing advertisements
- Managing entertainment content
- Managing watchlists
- Managing reviews and ratings
- Managing notifications
- Storing sports information
- Managing platform data

MVC Request Flow

The application follows the following request flow:

User → React Frontend → Express Routes → Controllers → Models → MongoDB Database

↓

Controller → JSON Response → React Frontend → User Interface

This architecture separates presentation, business logic, and data management, making the system easier to maintain, test, and extend.

5. Challenges

The development of NexPlay involves several technical and project management challenges. These challenges are expected during implementation and will be addressed through careful planning, proper software engineering practices, and Agile development.

5.1 Third-Party API Integration

The platform depends on external APIs such as TMDb API and Sports APIs to retrieve entertainment information, live sports scores, streaming availability, and match schedules. API rate limits, service downtime, and changes in API structures may temporarily affect system functionality.

5.2 Live Sports Data Synchronization

Displaying real-time sports scores and match updates requires continuous communication with external APIs. Ensuring low latency while minimizing unnecessary API requests is a significant technical challenge.

5.3 Recommendation System

Providing personalized entertainment recommendations requires analyzing user watchlists, browsing history, and preferences. Designing an efficient recommendation mechanism while maintaining good performance is an important challenge.

5.4 Secure Authentication and Authorization

The platform supports multiple user roles, including Administrators, Entertainment Companies, and Registered Users. Implementing secure authentication using JWT and enforcing Role-Based Access Control (RBAC) across all system modules is essential.

5.5 Responsive User Interface

The application must provide a consistent and user-friendly experience across desktops, laptops, tablets, and smartphones. Maintaining responsiveness for different screen sizes while ensuring high performance is an important design challenge.

5.6 Scalability

The system architecture should support increasing numbers of users, entertainment companies, advertisements, and entertainment content without requiring major architectural modifications.

5.7 Maintaining MVC Architecture

As the project grows, maintaining a clean separation between the Model, View, and Controller layers is essential for improving maintainability, testing, and future development.

6. Sprint Plan

The NexPlay project follows the Agile Software Development methodology. The implementation is divided into four development sprints, each focusing on a specific group of features.

Sprint 1 – Company & Platform Foundation

Objective

Develop the core platform infrastructure and enable entertainment companies to establish their presence on NexPlay.

Features

- Company Registration
- Company Profile Management
- Company Verification
- Company Dashboard
- Advertisement Management
- Campaign Management
- Upcoming Content Management

Deliverables

- Entertainment companies can create and manage company profiles.
- Administrators can verify company registrations.
- Companies can publish advertisements and promotional campaigns.
- Companies can publish upcoming entertainment content.

Sprint 2 – Entertainment Discovery

Objective

Develop entertainment browsing and discovery features for users.

Features

- Browse Entertainment Content
- Search Functionality
- Advanced Filtering
- Trending Content
- Featured Content
- Upcoming Release Calendar
- Content Details Page

Deliverables

- Users can browse entertainment content.
- Users can search and filter entertainment information.

- Detailed content pages are available.
- Trending and upcoming entertainment content is displayed.

Sprint 3 – Streaming Platform Integration

Objective

Connect entertainment information with official streaming platforms.

Features

- Streaming Platform Directory
- "Where to Watch" Feature
- Official Streaming Redirect
- Watchlist Management
- Personalized Recommendations

Deliverables

- Users can discover official streaming platforms.
- Watchlists are synchronized with user accounts.
- Personalized recommendations are generated.
- Secure redirection to authorized streaming services is implemented.

Sprint 4 – Sports & Community Features

Objective

Extend the platform with sports services and community engagement features.

Features

- Live Sports Dashboard
- Live Scores
- Match Schedule
- Official Sports Streaming Links
- Ratings and Reviews
- Notification System

Deliverables

- Live sports information is available.
- Official sports streaming links are provided.
- Users can submit ratings and reviews.
- Notifications are delivered for important platform events.

7. Acceptance Criteria

The NexPlay project shall be considered successfully completed when all of the following conditions are satisfied.

- All planned functional requirements have been successfully implemented.
- Company registration and verification operate correctly.
- Entertainment companies can manage advertisements and upcoming content.
- Users can browse entertainment content through categories.
- Advanced search and filtering function correctly.
- Trending and featured content is displayed dynamically.
- Official streaming platform redirects operate successfully.
- Personalized watchlists function correctly.
- Recommendation functionality provides relevant suggestions.
- Live sports information is retrieved successfully from external APIs.
- Ratings and reviews are stored and displayed correctly.
- Notification functionality operates correctly.
- Authentication and Role-Based Access Control (RBAC) function correctly.
- The application follows the MVC architectural pattern.
- RESTful APIs operate successfully.
- MongoDB stores and retrieves application data correctly.
- The application is responsive across desktop, tablet, and mobile devices.
- The project source code is maintained using GitHub.
- All planned Agile sprint deliverables are completed.
- The system passes functional and integration testing.

8. Conclusion

This Software Requirements Specification (SRS) presents a comprehensive blueprint for the development of **NexPlay**, a web-based Entertainment Discovery, Branding, and Marketing Platform developed using the MERN Stack.

The platform is designed to connect entertainment companies with audiences by providing a centralized environment for brand promotion, content discovery, official streaming platform integration, live sports information, personalized recommendations, and community engagement.

The proposed MVC architecture ensures modularity, maintainability, scalability, and efficient separation of concerns throughout the application. Integration with external APIs allows NexPlay to deliver accurate and up-to-date entertainment information while respecting copyright by redirecting users only to authorized streaming platforms.

By following Agile software development practices and implementing the functional and non-functional requirements defined in this document, NexPlay aims to provide a secure, responsive, and user-friendly platform capable of supporting future expansion and additional entertainment services.